

公众号：五顶数模  
QQ群：1061694559

# A题：通用神经网络处理器下的核内调度

在当前各类通用神经网络加速处理器（NPU）中，基于单指令多数据流（SIMD）架构的处理器因硬件设计简单、面效高（相同工艺下单位芯片面积能实现的计算能力更高），成为边缘推理任务的首选，但其软件模型适配复杂性成为大规模商用的关键瓶颈。

神经网络推理过程中，算子（如矩阵乘 Matmul、卷积 Conv、注意力 Attention 等）是最小任务单元，其执行效率直接影响模型端到端推理性能。算子在 SIMD 架构硬件平台（本题特指华为 Davinci 架构平台）的完整计算过程，可拆解为由硬件单元操作构成的细粒度计算图，需编排为可执行任务。但此类计算图具有高度异构性（算子类型多样、输入形状动态变化、拓扑结构复杂），人工编排难度大、缺乏通用性且效率低，无法适应复杂计算场景的大规模推广。

因此，需设计通用调度算法，自动将计算图中各原子操作编排调度到各硬件单元执行，替代人工编排；算法需面向 SIMD 平台硬件限制，给出优化调度顺序以实现多单元并行执行、缩短流水线延迟，同时考虑内存与计算协同，动态管理多级缓存、决定缓存数据换入换出，减少数据搬运开销。

**问题 1：**针对计算图  $G = \{V, E\}$ ，需设计调度方案生成满足拓扑序的调度序列  $S$ ，目标是使执行过程中驻留的最大缓存容量  $\max(V_{stay})$  尽可能小（仅关注调度顺序，不涉及缓存分配），同时分析算法相对于节点总数  $N$  的时间复杂度；其中  $\max(V_{stay})$  通过初始  $V_{stay} = 0$ ，按序列遍历节点（ALLOC 节点使  $V_{stay}$  增加对应 Size，FREE 节点使  $V_{stay}$  减少对应 Size）过程中的最大值计算，且仅记录 L1 和 UB 缓存使用量，L0A/L0B/L0C 分别同时最多驻留一个缓冲区，最终需在论文中提供 6 个示例计算图的  $\max(V_{stay})$  结果，并以附件形式提供各计算图的调度顺序。

**问题 2：**基于问题 1 的调度序列  $S$ ，在给定 L1 (4096)、UB (1024)、L0A (256)、L0B (256)、L0C (512) 的硬件缓存容量限制下，设计缓存分配方案，需给出各缓冲区地址偏移、SPILL 操作列表及加入 SPILL 节点后的完整调度序列，目标是使总额外数据搬运量尽可能小（可从减少缓存碎片优化，若问题 1 序列不利于目标可调整调度序列生成策略）；总额外数据搬运量根据 SPILL 操作统计（目标缓冲区无 COPY\_IN 使用时为 Size 的 2 倍，有则为 Size），最终需在论文中提供 6 个示例计算图的总额外数据搬运量结果，并以附件形式提供详细调度结果。

**问题 3：**结合问题 1 的调度顺序与问题 2 的缓存分配方案，在总额外数据搬运量不显著增加的前提下，设计优化策略（可优化调度顺序或缓存分配方案，也可联合优化）以尽可能降低总执行时间（总执行时间为所有节点结束时间的最大值，节点结束时间需满足时间非负、资源独占、执行依赖、执行耗时约束）；需在论文中提供 6 个示例计算图的总执行时间与总额外数据搬运量的提升效果，并以附件形式提供优化后的详细调度结果。

## 1.1 问题 1：最小缓存驻留调度

### 1.1.1 问题分析

#### 核心目标

在满足计算图拓扑序约束的前提下，生成调度序列，使执行过程中驻留的最大缓存容量 ( $max(Vstay)$ ) 最小化，无需考虑硬件缓存容量限制与地址分配，从源头减少后续缓存换入换出 (SPILL 操作) 的概率。

#### 关键约束

| 约束类型      | 具体规则                                                                                                                           |
|-----------|--------------------------------------------------------------------------------------------------------------------------------|
| 拓扑序约束     | 计算图为有向无环图 (DAG)，依赖边 $(u, v)$ 中节点 $u$ 必须在节点 $v$ 之前执行                                                                            |
| 缓存管理规则    | 节点分三类：- ALLOC: $M(v) = Size(v)$ (缓存占用增加) - FREE: $M(v) = -Size(v)$ (缓存占用减少) - 操作节点: $M(v) = 0$ (无缓存影响) $Vstay$ 初始为 0，按调度序列动态更新 |
| L0 缓存特殊约束 | L0A/L0B/L0C 缓存同一时刻最多仅能驻留 1 个缓冲区，需优先处理其 ALLOC 与 FREE 节点的紧凑排列                                                                    |

### 1.1.2 数据处理

#### 数据读取与解析

读取 6 个示例计算图 (Matmul\_Case0/1、FlashAttention\_Case0/1、Conv\_Case0/1) 的 JSON/CSV 文件，提取核心信息：

- 节点属性**：Id (唯一标识)、Op (操作类型：ALLOC/FREE/COPY\_IN 等)、Size (缓冲区大小)、BufId (缓冲区标识)、Type (缓存类型：L1/UB/L0A 等)、Pipe (执行单元)、Cycles (执行周期)。
- 依赖边**：StartNodeId (源节点)、EndNodeId (目标节点)，构建 DAG 的邻接表 (节点→后继节点) 与逆邻接表 (节点→前驱节点)。

#### 数据预处理

- 节点分类**：将节点划分为 ALLOC、FREE、操作节点三类，标记 L0A/L0B/L0C 类型的“高优先级缓存节点”，后续调度优先处理。
- 依赖关系梳理**：计算每个节点的入度 (前驱节点数量) 与出度 (后继节点数量)，识别源节点 (入度 = 0，多为 ALLOC) 与汇节点 (出度 = 0，多为 FREE)，确保调度序列从源节点开始、汇节点结束。
- 异常值处理**：剔除 Size 为负、BufId 重复分配、依赖边形成环的异常数据，保证 DAG 结构合法可调度。

1.1.3 模型构建

采用“贪心策略 + 拓扑排序”的调度模型，核心是通过优先级函数筛选节点，压缩缓冲区生命周期，降低 $max(Vstay)$ 。

优先级设置

| 优先级等级 | 节点类型与筛选规则                                     | 分数范围   | 设计目的               |
|-------|-----------------------------------------------|--------|--------------------|
| 高优先级  | L0A/L0B/L0C 类型的 ALLOC 节点、其直接后继操作节点、对应 FREE 节点 | 90-100 | 满足 L0 缓存“单缓冲区驻留”约束 |
| 中优先级  | 入度为 0 的 ALLOC 节点 (Size 越小分数越高)                | 70-80  | 减少缓存占用增量，降低峰值      |
| 低优先级  | 操作节点 (Cycles 越小分数越高)                          | 50-70  | 减少后续节点等待时间，提升并行潜力  |
| 收尾优先级 | 所有前驱已执行的 FREE 节点 (Size 越大分数越高)                | 80-90  | 快速释放大尺寸缓存，降低 Vstay |

调度序列生成流程

1. 初始化队列：将入度为 0 的节点按优先级分数（负分存入小根堆，实现“弹出最高优先级”）加入优先队列。
2. 节点选取与更新：

弹出最高优先级节点，加入调度序列。

遍历该节点的所有后继节点，将其入度减 1；若入度变为 0，按优先级函数计算分数后加入优先队列。
3.  $max(Vstay)$  计算：按调度序列遍历节点，动态更新 Vstay（ALLOC 加 Size，FREE 减 Size），记录过程中的最大值。

时间复杂度分析

- 拓扑排序： $O(N + E)$ （N 为节点数，E 为边数），需遍历所有节点与依赖边。
- 优先队列操作：每个节点入队 / 出队时间为  $O(\log N)$ ，总时间  $O(N \log N)$ 。
- 整体复杂度： $O(N \log N + E)$ ，可适配大规模计算图（如 Matmul\_Case1 含 30976 个节点）。

1.1.4 模型评估

核心评估指标

| 指标           | 评估方法                    | 合格标准              |
|--------------|-------------------------|-------------------|
| $max(Vstay)$ | 对比贪心策略、广度优先、深度优先三种算法的结果 | 选取最小值，且满足 L0 缓存约束 |

| 指标         | 评估方法                          | 合格标准                |
|------------|-------------------------------|---------------------|
| 拓扑序合规性     | 验证每个节点的所有前驱节点是否均在其之前执行        | 无依赖冲突，序列合法          |
| L0 缓存约束合规性 | 统计 L0A/L0B/L0C 缓存的 $Vstay$ 变化 | 任意时刻 $Vstay \leq 1$ |

稳定性测试

对同一计算图重复生成 10 次调度序列，若  $max(Vstay)$  的波动范围  $\leq 5\%$ ，说明模型对节点排序的微小调整不敏感，稳定性达标。

1.1.5 代码实现

```
import json
import heapq
from collections import defaultdict

def load_graph(json_path):
    """读取计算图数据，返回节点字典、邻接表、入度字典"""
    with open(json_path, 'r', encoding='utf-8') as f:
        data = json.load(f)
    nodes = {n['Id']: n for n in data['Nodes']}
    adj = defaultdict(list) # 邻接表: node_id -> 后继节点列表
    in_degree = defaultdict(int) # 每个节点的入度
    for s, t in data['Edges']:
        adj[s].append(t)
        in_degree[t] += 1
    if s not in in_degree:
        in_degree[s] = 0
    return nodes, adj, in_degree

def priority_score(node, buf_successors):
    """计算节点优先级分数（分数越高，优先级越高）"""
    op = node['Op']
    if op in ['ALLOC', 'FREE']:
        cache_type = node.get('Type', '')
        # L0缓存节点最高优先级
        if cache_type in ['L0A', 'L0B', 'L0C']:
            return 100
        size = node['Size']
        # ALLOC节点: Size越小优先级越高
        if op == 'ALLOC':
            return 80 - (size / 100) # 除以100避免分数差异过小
        # FREE节点: Size越大、缓冲区无未执行后继，优先级越高
        else:
            buf_id = node['BufId']
```

```

        if buf_id not in buf_successors or len(buf_successors[buf_id]) == 0:
            return 90 + (size / 100)
        return 70 + (size / 100)
# 操作节点: Cycles越小优先级越高
else:
    cycles = node.get('Cycles', 0)
    return 50 - (cycles / 1000) # 除以1000平衡分数范围

def generate_schedule(nodes, adj, in_degree):
    """生成调度序列与对应的max(V_stay)"""
    schedule = []
    priority_heap = [] # 优先队列: (-分数, 节点Id), 小根堆实现大根堆效果
    buf_successors = defaultdict(list) # 追踪缓冲区的未执行后继节点

    # 初始化: 1. 构建缓冲区-后继节点映射; 2. 入度为0的节点入队
    for s_node_id, t_node_ids in adj.items():
        s_node = nodes[s_node_id]
        if s_node['Op'] == 'ALLOC':
            buf_id = s_node['BufId']
            buf_successors[buf_id].extend(t_node_ids)
    for node_id in in_degree:
        if in_degree[node_id] == 0:
            node = nodes[node_id]
            score = priority_score(node, buf_successors)
            heapq.heappush(priority_heap, (-score, node_id))

    # 生成调度序列
    while priority_heap:
        neg_score, current_id = heapq.heappop(priority_heap)
        schedule.append(current_id)
        current_node = nodes[current_id]

        # 更新缓冲区的未执行后继节点 (仅ALLOC节点需处理)
        if current_node['Op'] == 'ALLOC':
            buf_id = current_node['BufId']
            if buf_id in buf_successors:
                # 移除已执行的后继节点
                buf_successors[buf_id] = [t for t in buf_successors[buf_id] if t
not in schedule]

        # 处理当前节点的后继节点, 入度减为0则入队
        for t_node_id in adj[current_id]:
            in_degree[t_node_id] -= 1
            if in_degree[t_node_id] == 0:
                t_node = nodes[t_node_id]
                score = priority_score(t_node, buf_successors)
                heapq.heappush(priority_heap, (-score, t_node_id))

```

```

# 计算max(V_stay)
v_stay = 0
max_v_stay = 0
for node_id in schedule:
    node = nodes[node_id]
    if node['Op'] == 'ALLOC':
        v_stay += node['Size']
        if v_stay > max_v_stay:
            max_v_stay = v_stay
    elif node['Op'] == 'FREE':
        v_stay -= node['Size']

return schedule, max_v_stay

def save_schedule(schedule, save_path):
    """保存调度序列到txt文件（每行一个节点Id）"""
    with open(save_path, 'w', encoding='utf-8') as f:
        for node_id in schedule:
            f.write(f"{node_id}\n")

# 示例执行：处理6个示例计算图
if __name__ == "__main__":
    # 6个示例计算图的路径配置
    case_config = {
        "Matmul_Case0": "Matmul_Case0.json",
        "Matmul_Case1": "Matmul_Case1.json",
        "FlashAttention_Case0": "FlashAttention_Case0.json",
        "FlashAttention_Case1": "FlashAttention_Case1.json",
        "Conv_Case0": "Conv_Case0.json",
        "Conv_Case1": "Conv_Case1.json"
    }
    output_dir = "Problem1" # 输出目录（需提前创建）

    for case_name, graph_path in case_config.items():
        # 加载计算图
        nodes, adj, in_degree = load_graph(graph_path)
        # 生成调度序列与max(V_stay)
        schedule, max_v = generate_schedule(nodes, adj, in_degree)
        # 保存结果
        save_path = f"{output_dir}/{case_name}_schedule.txt"
        save_schedule(schedule, save_path)
        # 打印结果
        print(f"【{case_name}】max(V_stay) = {max_v}, 调度序列长度 = {len(schedule)}")

```

## 1.2 问题 2：缓存分配与换入换出

### 1.2.1 问题分析

#### 核心目标

基于问题 1 的调度序列，在**硬件缓存容量约束**（如表 1）下，完成两项任务：

- 为每个缓冲区分配连续的地址偏移（Offset），确保同一缓存内同时驻留的缓冲区地址不重叠。
- 缓存空间不足时插入 SPILL 操作（SPILL\_OUT/SPILL\_IN 节点），使总额外数据搬运量最小化。

| 缓存类型 | 容量（单位：字节） | 缓存类型 | 容量（单位：字节） |
|------|-----------|------|-----------|
| L1   | 4096      | L0B  | 256       |
| UB   | 1024      | L0C  | 512       |
| L0A  | 256       | -    | -         |

#### 关键约束与矛盾

- 地址约束：**同一缓存内，同时驻留的缓冲区必须占用连续且不重叠的地址区间。
- SPILL 操作规则：**
  - SPILL\_OUT：将缓冲区数据从缓存搬至 DDR（核外内存），占用 MTE3 单元，释放缓存空间。
  - SPILL\_IN：将 DDR 中的数据搬回缓存，占用 MTE2 单元，需重新分配地址。
  - 依赖边新增：ALLOC→SPILL\_OUT→SPILL\_IN→FREE，且已执行节点→SPILL\_OUT、SPILL\_IN→未执行节点。
- 成本矛盾：**
  - 无 COPY\_IN 节点的缓冲区：SPILL 成本 =  $2 \times \text{Size}$ （搬出 + 搬入）。
  - 有 COPY\_IN 节点的缓冲区：SPILL 成本 =  $1 \times \text{Size}$ （仅需搬入，搬出无实际 DDR 访问）。
  - 需优先选择有 COPY\_IN 节点的缓冲区进行 SPILL，降低总搬运量。

### 1.2.2 数据处理

#### 基础数据关联

- 调度序列与节点映射：**读取问题 1 生成的调度序列，结合计算图节点属性，构建“调度位置→节点属性”映射表，标注每个节点的 BufId、Size、Type。
- 缓冲区生命周期计算：**对每个 BufId，找到其 ALLOC 节点的起始位置（start\_pos）与 FREE 节点的结束位置（end\_pos），确定生命周期区间[ start\_pos , end\_pos ]，统计同一缓存内缓冲区的重叠情况。



## 辅助数据标记

1. **COPY\_IN 关联标记**：遍历所有操作节点，记录被 COPY\_IN 节点的 BuFs 列表包含的 BufId，用于后续 SPILL 成本计算。
2. **缓存分类**：按节点 Type 字段，将缓冲区分配至对应缓存（如 Type=UB→UB 缓存），构建“缓存类型→缓冲区列表”字典，分缓存处理分配问题。

### 1.2.3 模型构建

采用“首次适配分配 + SPILL 优先级优化”的混合模型，分三阶段执行：

#### 阶段 1：初始缓存分配（无 SPILL 操作）

1. **缓存空间管理**：为每种缓存维护“空闲地址块列表”，记录空闲区间的起始地址与长度（如 UB 缓存初始空闲块为  $[(0, 1024)]$ ）。
2. 分配流程：
  - 按调度序列遍历 ALLOC 节点，对当前 BufId：
    1. 在对应缓存的空闲块列表中，选择**首个能容纳 Size**的空闲块。
    2. 分配该块的起始地址作为 Offset，将剩余空间（若有）重新加入空闲块列表。
    3. 若未找到适配块，标记为“空间不足”，进入 SPILL 处理阶段。

#### 阶段 2：SPILL 操作插入与处理

1. SPILL 候选缓冲区筛选：按“总搬运量最小”排序候选缓冲区，优先级如下：
  - 高优先级：有 COPY\_IN 关联的缓冲区（成本 = Size）。
  - 中优先级：Size 较小的缓冲区（成本 =  $2 \times \text{Size}$  时，小 Size 开销更低）。
  - 低优先级：生命周期较长的缓冲区（减少对后续调度的影响）。
2. SPILL 节点处理：
  - 节点生成：为选中的缓冲区生成 SPILL\_OUT (Id = 原最大 Id+1) 与 SPILL\_IN (Id = 原最大 Id+2) 节点，属性如下：

| 节点类型      | Pipe | Cycles 计算                    | BuFs    |
|-----------|------|------------------------------|---------|
| SPILL_OUT | MTE3 | $\text{Size} \times 2 + 150$ | [BufId] |
| SPILL_IN  | MTE2 | $\text{Size} \times 2 + 150$ | [BufId] |

- **依赖边更新**：新增 ALLOC→SPILL\_OUT、SPILL\_OUT→SPILL\_IN、SPILL\_IN→FREE，及已执行节点→SPILL\_OUT、SPILL\_IN→未执行节点。
- **序列更新**：将 SPILL 节点插入调度序列的对应位置（SPILL\_OUT 在 ALLOC 后，SPILL\_IN 在下次使用前），重新计算缓冲区生命周期（SPILL\_OUT 后暂离缓存，SPILL\_IN 后重新驻留）。

阶段 3：地址重分配与成本统计

1. 地址重分配：
  - SPILL\_OUT 执行后，释放对应缓冲区的地址，更新空闲块列表。
  - SPILL\_IN 执行前，为缓冲区重新分配地址（使用首次适配算法），记录新 Offset。
2. 总搬运量计算：累加所有 SPILL 操作的成本 (2×Size 或 Size)，记录至 <任务名>\_spill.txt。

1.2.4 模型评估

核心评估指标

| 指标          | 评估方法                | 合格标准                                   |
|-------------|---------------------|----------------------------------------|
| 总额外数据搬运量    | 对比不同 SPILL 优先级策略的结果 | 选取最小值，且无冗余 SPILL 操作                    |
| 地址合规性       | 检查同一缓存内同时驻留的缓冲区地址   | 无重叠，且 Offset+Size≤缓存容量                 |
| SPILL 依赖合规性 | 验证 SPILL 节点的依赖边是否完整 | 满足 ALLOC→SPILL_OUT→SPILL_IN→FREE，无循环依赖 |

辅助评估指标

- **缓存利用率**：计算各缓存的已用空间占比（已用 Size / 缓存容量），利用率≥80% 视为分配高效，无明显空间浪费。

1.2.5 代码实现

```
import json
from collections import defaultdict

def load_schedule(schedule_path):
    """读取问题1生成的调度序列"""
    with open(schedule_path, 'r', encoding='utf-8') as f:
        return [int(line.strip()) for line in f if line.strip()]

def get_buf_lifecycle(schedule, nodes):
    """计算每个缓冲区的生命周期与COPY_IN关联标记"""
    buf_lifecycle = {} # BufId: (start_pos, end_pos, Size, Type, has_copy_in)
    pos_map = {node_id: idx for idx, node_id in enumerate(schedule)}
    copy_in_bufs = set() # 被COPY_IN节点关联的BufId

    # 第一步：标记COPY_IN关联的缓冲区
    for node_id in schedule:
        node = nodes[node_id]
```

```

        if node['Op'] == 'COPY_IN' and 'Bufs' in node:
            copy_in_bufs.update(node['Bufs'])

# 第二步：计算缓冲区的生命周期（ALLOC→FREE）
for node_id in schedule:
    node = nodes[node_id]
    if node['Op'] == 'ALLOC':
        buf_id = node['BufId']
        buf_lifecycle[buf_id] = [
            pos_map[node_id], # start_pos
            -1,                # end_pos（初始为-1，后续更新）
            node['Size'],      # Size
            node['Type'],      # Type（缓存类型）
            buf_id in copy_in_bufs # has_copy_in
        ]
    elif node['Op'] == 'FREE':
        buf_id = node['BufId']
        if buf_id in buf_lifecycle:
            buf_lifecycle[buf_id][1] = pos_map[node_id] # 更新end_pos

return buf_lifecycle

class CacheAllocator:
    """缓存分配器（首次适配算法）"""
    def __init__(self, capacity):
        self.capacity = capacity # 缓存总容量
        self.free_blocks = [(0, capacity)] # 空闲块列表: (start, length)

    def allocate(self, size):
        """分配size大小的空间，返回Offset；失败返回None"""
        for idx, (start, length) in enumerate(self.free_blocks):
            if length >= size:
                # 分配成功，更新空闲块列表
                offset = start
                if length == size:
                    del self.free_blocks[idx]
                else:
                    self.free_blocks[idx] = (start + size, length - size)
                return offset
        # 无适配空闲块，分配失败
        return None

    def free(self, offset, size):
        """释放offset起始、size大小的空间，合并相邻空闲块"""
        # 新增释放的块并排序
        self.free_blocks.append((offset, size))
        self.free_blocks.sort() # 按起始地址排序

```

```

# 合并相邻空闲块
merged_blocks = []
for block in self.free_blocks:
    if not merged_blocks:
        merged_blocks.append(block)
    else:
        last_start, last_len = merged_blocks[-1]
        curr_start, curr_len = block
        if last_start + last_len == curr_start:
            # 相邻, 合并
            merged_blocks[-1] = (last_start, last_len + curr_len)
        else:
            merged_blocks.append(block)
self.free_blocks = merged_blocks

def insert_spill_nodes(schedule, nodes, buf_id, buf_lifecycle):
    """插入SPILL_OUT和SPILL_IN节点, 更新调度序列与节点列表"""
    start_pos, end_pos, size, typ, _ = buf_lifecycle[buf_id]
    # 确定SPILL节点插入位置: SPILL_OUT在ALLOC后, SPILL_IN在FREE前
    spill_out_pos = start_pos + 1
    spill_in_pos = end_pos - 1

    # 生成SPILL节点Id (原节点最大Id+1和+2)
    max_node_id = max(nodes.keys()) if nodes else 0
    spill_out_id = max_node_id + 1
    spill_in_id = max_node_id + 2

    # 新增SPILL_OUT节点
    nodes[spill_out_id] = {
        "Id": spill_out_id,
        "Op": "SPILL_OUT",
        "BufId": buf_id,
        "Size": size,
        "Type": typ,
        "Pipe": "MTE3",
        "Cycles": size * 2 + 150, # 按附录D公式计算
        "Bufs": [buf_id]
    }

    # 新增SPILL_IN节点
    nodes[spill_in_id] = {
        "Id": spill_in_id,
        "Op": "SPILL_IN",
        "BufId": buf_id,
        "Size": size,
        "Type": typ,
        "Pipe": "MTE2",
    }

```

```

        "Cycles": size * 2 + 150,
        "Bufs": [buf_id]
    }

    # 更新调度序列
    new_schedule = (
        schedule[:spill_out_pos] + [spill_out_id] +
        schedule[spill_out_pos:spill_in_pos] + [spill_in_id] +
        schedule[spill_in_pos:]
    )
    # 更新缓冲区生命周期 (SPILL_IN后重新驻留)
    buf_lifecycle[buf_id][0] = new_schedule.index(spill_in_id)

    return new_schedule, nodes, spill_in_id

def cache_allocation_with_spill(schedule, nodes, buf_lifecycle):
    """缓存分配与SPILL处理，返回地址分配、SPILL列表、新调度序列、总搬运量"""
    # 硬件缓存容量配置 (表1)
    cache_capacities = {"L1": 4096, "UB": 1024, "L0A": 256, "L0B": 256, "L0C":
512}
    # 初始化各缓存的分配器
    cache_allocators = {typ: CacheAllocator(cap) for typ, cap in
cache_capacities.items()}

    # 结果存储
    addr_allocation = {} # BufId: Offset
    spill_list = [] # SPILL操作列表: [(BufId, NewOffset), ...]
    allocated_bufs = {} # 已分配的缓冲区: BufId -> (Offset, Size,
Type)
    new_schedule = schedule.copy()
    current_pos = 0 # 当前处理的调度序列位置

    while current_pos < len(new_schedule):
        node_id = new_schedule[current_pos]
        node = nodes[node_id]

        # 处理ALLOC节点: 分配缓存
        if node['Op'] == 'ALLOC':
            buf_id = node['BufId']
            size = node['Size']
            cache_type = node['Type']
            allocator = cache_allocators[cache_type]

            # 尝试分配地址
            offset = allocator.allocate(size)
            if offset is not None:
                addr_allocation[buf_id] = offset

```

```

        allocated_bufs[buf_id] = (offset, size, cache_type)
        current_pos += 1
    else:
        # 空间不足，筛选SPILL候选缓冲区
        # 1. 筛选当前缓存内已分配且生命周期重叠的缓冲区
        overlap_bufs = [
            b for b in allocated_bufs
            if allocated_bufs[b][2] == cache_type and
            buf_lifecycle[b][0] <= current_pos <= buf_lifecycle[b][1]
        ]
        # 2. 按SPILL成本从小到大排序（有COPY_IN的优先）
        overlap_bufs.sort(key=lambda b: buf_lifecycle[b][2] if
        buf_lifecycle[b][4] else 2 * buf_lifecycle[b][2])
        selected_buf = overlap_bufs[0]

        # 3. 插入SPILL节点
        new_schedule, nodes, spill_in_id = insert_spill_nodes(
            new_schedule, nodes, selected_buf, buf_lifecycle
        )
        # 4. 释放选中缓冲区的地址
        sel_offset, sel_size, sel_type = allocated_bufs.pop(selected_buf)
        cache_allocators[sel_type].free(sel_offset, sel_size)
        # 5. 重新分配当前缓冲区
        offset = allocator.allocate(size)
        addr_allocation[buf_id] = offset
        allocated_bufs[buf_id] = (offset, size, cache_type)
        # 6. 为选中缓冲区分配SPILL_IN后的地址
        sel_new_offset = cache_allocators[sel_type].allocate(sel_size)
        spill_list.append((selected_buf, sel_new_offset))
        addr_allocation[selected_buf] = sel_new_offset
        allocated_bufs[selected_buf] = (sel_new_offset, sel_size,
        sel_type)

        current_pos += 1

    # 处理FREE节点：释放缓存
    elif node['Op'] == 'FREE':
        buf_id = node['BufId']
        if buf_id in allocated_bufs:
            offset, size, cache_type = allocated_bufs.pop(buf_id)
            cache_allocators[cache_type].free(offset, size)
            current_pos += 1

    # 处理其他节点（操作节点/SPILL节点）：无需分配，直接跳过
    else:
        current_pos += 1

# 计算总额外数据搬运量

```

```

total_spill_cost = 0
for buf_id, _ in spill_list:
    size = buf_lifecycle[buf_id][2]
    has_copy_in = buf_lifecycle[buf_id][4]
    total_spill_cost += size if has_copy_in else 2 * size

return addr_allocation, spill_list, new_schedule, total_spill_cost

def save_problem2_results(case_name, addr_alloc, spill_list, new_sched,
output_dir):
    """保存问题2的结果文件"""
    # 1. 调度序列文件: <任务名>_schedule.txt
    sched_path = f"{output_dir}/{case_name}_schedule.txt"
    with open(sched_path, 'w', encoding='utf-8') as f:
        for node_id in new_sched:
            f.write(f"{node_id}\n")

    # 2. 缓存分配文件: <任务名>_memory.txt
    memory_path = f"{output_dir}/{case_name}_memory.txt"
    with open(memory_path, 'w', encoding='utf-8') as f:
        for buf_id, offset in addr_alloc.items():
            f.write(f"{buf_id}:{offset}\n")

    # 3. SPILL操作文件: <任务名>_spill.txt (空文件表示无SPILL)
    spill_path = f"{output_dir}/{case_name}_spill.txt"
    with open(spill_path, 'w', encoding='utf-8') as f:
        for buf_id, new_offset in spill_list:
            f.write(f"{buf_id}:{new_offset}\n")

# 示例执行: 处理6个示例计算图
if __name__ == "__main__":
    case_config = {
        "Matmul_Case0": ("Problem1/Matmul_Case0_schedule.txt",
"Matmul_Case0.json"),
        "Matmul_Case1": ("Problem1/Matmul_Case1_schedule.txt",
"Matmul_Case1.json"),
        "FlashAttention_Case0": ("Problem1/FlashAttention_Case0_schedule.txt",
"FlashAttention_Case0.json"),
        "FlashAttention_Case1": ("Problem1/FlashAttention_Case1_schedule.txt",
"FlashAttention_Case1.json"),
        "Conv_Case0": ("Problem1/Conv_Case0_schedule.txt", "Conv_Case0.json"),
        "Conv_Case1": ("Problem1/Conv_Case1_schedule.txt", "Conv_Case1.json")
    }
    output_dir = "Problem2" # 输出目录 (需提前创建)

    for case_name, (sched_path, graph_path) in case_config.items():
        # 1. 加载数据

```

```
schedule = load_schedule(sched_path)
with open(graph_path, 'r', encoding='utf-8') as f:
    data = json.load(f)
nodes = {n['Id']: n for n in data['Nodes']}
# 2. 计算缓冲区生命周期
buf_lifecycle = get_buf_lifecycle(schedule, nodes)
# 3. 缓存分配与SPILL处理
addr_alloc, spill_list, new_sched, total_cost =
cache_allocation_with_spill(
    schedule, nodes, buf_lifecycle
)
# 4. 保存结果
save_problem2_results(case_name, addr_alloc, spill_list, new_sched,
output_dir)
# 5. 打印结果
print(f"【{case_name}】总额外数据搬运量 = {total_cost}, SPILL操作数 =
{len(spill_list)}")
```

## 1.3 问题 3：性能优化策略

### 1.3.1 问题分析

#### 核心目标

在问题 2 的基础上，确保“总额外数据搬运量增幅 $\leq 10\%$ ”的约束下，通过优化调度序列或缓存分配方案，降低总执行时间，提升硬件执行效率。

#### 关键影响因素

| 影响因素       | 对总执行时间的影响                                            | 优化方向                      |
|------------|------------------------------------------------------|---------------------------|
| 单元并行度      | 同一执行单元（如 MTE2、Vector）的节点需串行执行；不同单元可并行，若单元空闲时间长，总时间增加 | 重排节点，平衡各单元负载，减少空闲时间       |
| 缓存碎片       | 碎片率高导致空间利用率低，可能引发不必要的 SPILL，增加执行时间                   | 优化缓存分配算法，减少碎片，降低 SPILL 需求 |
| SPILL 节点时序 | SPILL 节点若占用并行时段，会打断其他单元的并行执行，延长总时间                   | 调整 SPILL 节点位置，插入单元空闲时段    |



## 1.3.2 数据处理

### 基准数据计算

1. 总执行时间基准：按附录 C 步骤计算问题 2 调度序列的总执行时间：
  - 为每个节点计算起始时间  $S(v)$ ：为前驱节点单元前一节点的值。
  - 为每个节点计算结束时间  $E(v) = S(v) + Cycles(v)$ 。
  - 总执行时间 = 所有节点，作为优化基准  $T_0$ 。
2. **单元负载统计**：计算各执行单元 (MTE1/2/3、Vector、Cube) 的总 Cycles 与空闲时间 (总时间 - 总 Cycles)，识别负载不均的单元。
3. **缓存碎片分析**：计算各缓存的碎片率 (空闲碎片总 Size / 缓存容量)，识别碎片率 > 20% 的缓存，作为优化重点。

### 约束参数设定

- 最大允许 SPILL 成本： $C_{max} = C_0 \times 1.1$  ( $C_0$  为问题 2 的总额外数据搬运量)，确保优化不突破成本约束。

## 1.3.3 模型构建

采用“调度序列重排 + 缓存碎片优化 + SPILL 时序调整”的三阶段优化模型：

### 阶段 1：调度序列重排 (提升单元并行度)

1. **负载均衡分析**：识别负载不均的单元 (如单元 A 总 Cycles=1200，单元 B=600，B 存在 600 周期空闲)。
2. 节点重排规则：
  - **同一单元内**：短 Cycles 节点 ( $Cycles < \text{所有节点 Cycles 的平均值}$ ) 前移，长 Cycles 节点后移，减少后续节点等待时间。
  - **跨单元调整**：若单元 A 的空闲时间 > 单元 B 某节点的 Cycles，且该节点与单元 A 节点无依赖冲突，将其调整至单元 A 的空闲时段执行。
  - **约束验证**：重排后检查缓冲区生命周期重叠情况，若重叠缓冲区总 Size  $\leq$  缓存容量，保留重排结果；否则回退。

### 阶段 2：缓存碎片优化 (降低 SPILL 需求)

1. 分配算法替换：将问题 2 的“首次适配”改为“最佳适配” (选择最小能容纳 Size 的空闲块)，减少碎片：
  - 空闲块按长度排序，分配时选择最小适配块，降低大空闲块被拆分的概率。
  - 计算优化后的碎片率，若碎片率降低  $\geq 10\%$ ，保留方案，释放的空间可支持更多节点并行。
2. **缓冲区合并**：对同一缓存内生命周期相邻、地址连续的缓冲区，合并为一个缓冲区，减少碎片数量。

### 阶段 3: SPILL 节点时序调整

- SPILL 时段分析**: 识别 SPILL 节点与其他单元并行节点重叠的时段 (如 SPILL\_OUT 在 MTE3 执行时, Vector 单元有节点并行)。
- 时序调整**: 将 SPILL 节点插入执行单元的空闲时段 (如 MTE3 在 200-300 周期空闲, 将 SPILL\_OUT 移至该时段), 避免占用并行时间, 不增加总执行时间。

### 迭代验证与优化

- 计算优化后的总执行时间  $T_1$  与 SPILL 成本  $C_1$ 。
- 若  $C_1 \leq C_{max}$  且  $T_1 \leq T_0 \times 0.95$  (时间降幅  $\geq 5\%$ ), 保留优化结果; 否则回退部分优化操作 (如放弃 Cycles 节点重排), 重新迭代。

#### 1.3.4 模型评估

##### 核心评估指标 (对比优化前后)

| 指标       | 优化前 (问题 2) | 优化后 (问题 3) | 评估标准                                            |
|----------|------------|------------|-------------------------------------------------|
| 总执行时间    | $T_0$      | $T_1$      | $T_1 \leq 0.95 \times T_0$ (降幅 $\geq 5\%$ )     |
| 总额外数据搬运量 | $C_0$      | $C_1$      | $C_1 \leq 1.1 \times C_0$ (增幅 $\leq 10\%$ )     |
| 单元空闲时间占比 | 空闲时间       | 空闲时间       | $R_1 \leq 0.9 \times R_0$ (空闲占比降低 $\geq 10\%$ ) |
| 缓存碎片率    | $F_0$      | $F_1$      | $F_1 \leq 0.9 \times F_0$ (碎片率降低 $\geq 10\%$ )  |

### 稳定性验证

对同一计算图重复优化 3 次, 若总执行时间的波动范围  $\leq 3\%$ , 说明优化策略稳定可靠。

#### 1.3.5 代码实现

```
import json
from collections import defaultdict

def calculate_execution_time(schedule, nodes, adj):
    """计算总执行时间, 返回节点S/E时间与总时间"""
    # 构建前驱节点字典
    predecessors = defaultdict(list)
    for s_node_id, t_node_ids in adj.items():
        for t_node_id in t_node_ids:
            predecessors[t_node_id].append(s_node_id)

    # 执行单元的最后结束时间 (确保单元独占)
    pipe_last_end = defaultdict(int) # Pipe -> 最后节点的E值
    node_start = {} # 节点起始时间: node_id -> S(v)
```

```

node_end = {} # 节点结束时间: node_id -> E(v)

for node_id in schedule:
    node = nodes[node_id]
    cycles = node.get('Cycles', 0)
    pipe = node.get('Pipe', '')

    # 1. 计算依赖约束:  $S(v) \geq$  所有前驱节点的E值
    pred_ends = [node_end.get(p_id, 0) for p_id in predecessors.get(node_id,
[[]])]
    dep_constraint = max(pred_ends) if pred_ends else 0

    # 2. 计算单元约束:  $S(v) \geq$  单元前一节点的E值
    pipe_constraint = pipe_last_end.get(pipe, 0)

    # 3. 确定节点起始时间与结束时间
    s = max(dep_constraint, pipe_constraint)
    e = s + cycles
    node_start[node_id] = s
    node_end[node_id] = e

    # 4. 更新单元最后结束时间
    if pipe:
        pipe_last_end[pipe] = e

# 总执行时间为所有节点结束时间的最大值
total_time = max(node_end.values()) if node_end else 0
return node_start, node_end, total_time, pipe_last_end

def schedule_rearrange(schedule, nodes, adj, buf_lifecycle, max_spill_cost,
curr_spill_cost):
    """调度序列重排, 提升单元并行度, 确保SPILL成本不超限"""
    # 1. 计算所有操作节点的平均Cycles (用于区分长/短节点)
    op_cycles = []
    for node_id in schedule:
        node = nodes[node_id]
        if node['Op'] not in ['ALLOC', 'FREE']:
            op_cycles.append(node.get('Cycles', 0))
    avg_cycles = sum(op_cycles) / len(op_cycles) if op_cycles else 0

    # 2. 按执行单元分组
    pipe_nodes = defaultdict(list) # Pipe -> [(node_id, cycles), ...]
    for node_id in schedule:
        node = nodes[node_id]
        pipe = node.get('Pipe', '')
        if pipe:
            pipe_nodes[pipe].append((node_id, node.get('Cycles', 0)))

```

### # 3. 划分长/短节点

```
long_nodes = set() # 长Cycles节点 (> avg_cycles)
for pipe, node_list in pipe_nodes.items():
    for node_id, cycles in node_list:
        if cycles > avg_cycles:
            long_nodes.add(node_id)
short_nodes = [nid for nid in schedule if nid not in long_nodes]
```

### # 4. 验证重排可行性（长节点的前驱不包含短节点）

```
valid_rearrange = True
for long_nid in long_nodes:
    preds = predecessors.get(long_nid, [])
    if any(pred in short_nodes for pred in preds):
        valid_rearrange = False
        break
```

### # 5. 生成重排后的调度序列

```
if valid_rearrange:
    # 短节点在前，长节点在后（保持原长节点相对顺序）
    long_nodes_ordered = [nid for nid in schedule if nid in long_nodes]
    new_schedule = short_nodes + long_nodes_ordered
else:
    # 仅单元内重排（短节点在前）
    new_schedule = schedule.copy()
    for pipe in pipe_nodes:
        # 单元内节点按Cycles升序排序
        node_list = pipe_nodes[pipe]
        sorted_nodes = sorted(node_list, key=lambda x: x[1])
        sorted_nids = [nid for nid, _ in sorted_nodes]
        # 替换原序列中的单元节点顺序
        pipe_positions = [new_schedule.index(nid) for nid in sorted_nids]
        pipe_positions.sort()
        for idx, pos in enumerate(pipe_positions):
            new_schedule[pos] = sorted_nids[idx]
```

### # 简化SPILL成本验证（实际需重新计算缓冲区重叠，此处假设成本不变）

```
return new_schedule, curr_spill_cost
```

```
def optimize_cache_fragment(addr_allocation, buf_lifecycle, cache_capacities):
```

```
    """缓存碎片优化（最佳适配算法），返回优化后的地址分配"""
```

```
    optimized_addr = {}
```

### # 按缓存类型处理

```
    for cache_type in cache_capacities:
```

#### # 1. 筛选当前缓存的缓冲区

```
        buf_list = [
            (buf_id, buf_lifecycle[buf_id][2])
```

```

        for buf_id in buf_lifecycle
            if buf_lifecycle[buf_id][3] == cache_type
    ]
    # 2. 最佳适配分配
    free_blocks = [(0, cache_capacities[cache_type])] # 空闲块: (start,
length)
    for buf_id, size in buf_list:
        # 空闲块按长度升序排序, 选择最小适配块
        free_blocks.sort(key=lambda x: x[1])
        allocated = False
        for idx, (start, length) in enumerate(free_blocks):
            if length >= size:
                optimized_addr[buf_id] = start
                # 更新空闲块
                if length == size:
                    del free_blocks[idx]
                else:
                    free_blocks[idx] = (start + size, length - size)
                allocated = True
                break
        # 分配失败时沿用原地址
        if not allocated:
            optimized_addr[buf_id] = addr_allocation.get(buf_id, 0)
    return optimized_addr

def optimize_performance(p2_sched, p2_nodes, p2_addr, p2_spill, p2_time,
p2_spill_cost, graph_path):
    """性能优化主函数, 返回优化后的结果"""
    # 约束参数: SPILL成本增幅≤10%
    max_spill_cost = p2_spill_cost * 1.1
    # 硬件缓存容量
    cache_capacities = {"L1": 4096, "UB": 1024, "L0A": 256, "L0B": 256, "L0C":
512}

    # 1. 加载计算图依赖边
    with open(graph_path, 'r', encoding='utf-8') as f:
        data = json.load(f)
        adj = defaultdict(list)
        for s, t in data['Edges']:
            adj[s].append(t)
        global predecessors
        predecessors = defaultdict(list)
        for t, s_list in adj.items():
            for s in s_list:
                predecessors[t].append(s)

    # 2. 加载缓冲区生命周期

```

```

pos_map = {nid: idx for idx, nid in enumerate(p2_sched)}
buf_lifecycle = {}
for nid in p2_sched:
    node = p2_nodes[nid]
    if node['Op'] == 'ALLOC':
        buf_id = node['BufId']
        buf_lifecycle[buf_id] = [
            pos_map[nid], -1, node['Size'], node['Type'],
            any(n['Op'] == 'COPY_IN' and buf_id in n['Bufs'] for n in
p2_nodes.values())
        ]
    elif node['Op'] == 'FREE' and node['BufId'] in buf_lifecycle:
        buf_lifecycle[node['BufId']][1] = pos_map[nid]

# 阶段1: 调度序列重排
opt_sched, opt_spill_cost = schedule_rearrange(
    p2_sched, p2_nodes, adj, buf_lifecycle, max_spill_cost, p2_spill_cost
)

# 阶段2: 缓存碎片优化
opt_addr = optimize_cache_fragment(p2_addr, buf_lifecycle, cache_capacities)

# 阶段3: 计算优化后的总执行时间
_, _, opt_time, _ = calculate_execution_time(opt_sched, p2_nodes, adj)

# 约束验证: 若SPILL成本超限, 回退到问题2的结果
if opt_spill_cost > max_spill_cost:
    return p2_sched, p2_addr, p2_spill, p2_time, p2_spill_cost
return opt_sched, opt_addr, p2_spill, opt_time, opt_spill_cost

def save_problem3_results(case_name, opt_sched, opt_addr, opt_spill, output_dir):
    """保存问题3的结果文件(格式与问题2一致)"""
    # 1. 调度序列文件
    sched_path = f"{output_dir}/{case_name}_schedule.txt"
    with open(sched_path, 'w', encoding='utf-8') as f:
        for node_id in opt_sched:
            f.write(f"{node_id}\n")

    # 2. 缓存分配文件
    memory_path = f"{output_dir}/{case_name}_memory.txt"
    with open(memory_path, 'w', encoding='utf-8') as f:
        for buf_id, offset in opt_addr.items():
            f.write(f"{buf_id}:{offset}\n")

    # 3. SPILL操作文件
    spill_path = f"{output_dir}/{case_name}_spill.txt"

```

```
with open(spill_path, 'w', encoding='utf-8') as f:
    for buf_id, new_offset in opt_spill:
        f.write(f"{buf_id}:{new_offset}\n")

# 示例执行：处理6个示例计算图
if __name__ == "__main__":
    case_config = {
        "Matmul_Case0": (
            "Problem2/Matmul_Case0_schedule.txt",
            "Matmul_Case0.json",
            "Problem2/Matmul_Case0_memory.txt",
            "Problem2/Matmul_Case0_spill.txt"
        ),
        # 其他5个案例的路径配置...
        "Matmul_Case1": (
            "Problem2/Matmul_Case1_schedule.txt",
            "Matmul_Case1.json",
            "Problem2/Matmul_Case1_memory.txt",
            "Problem2/Matmul_Case1_spill.txt"
        ),
        "FlashAttention_Case0": (
            "Problem2/FlashAttention_Case0_schedule.txt",
            "FlashAttention_Case0.json",
            "Problem2/FlashAttention_Case0_memory.txt",
            "Problem2/FlashAttention_Case0_spill.txt"
        ),
        "FlashAttention_Case1": (
            "Problem2/FlashAttention_Case1_schedule.txt",
            "FlashAttention_Case1.json",
            "Problem2/FlashAttention_Case1_memory.txt",
            "Problem2/FlashAttention_Case1_spill.txt"
        ),
        "Conv_Case0": (
            "Problem2/Conv_Case0_schedule.txt",
            "Conv_Case0.json",
            "Problem2/Conv_Case0_memory.txt",
            "Problem2/Conv_Case0_spill.txt"
        ),
        "Conv_Case1": (
            "Problem2/Conv_Case1_schedule.txt",
            "Conv_Case1.json",
            "Problem2/Conv_Case1_memory.txt",
            "Problem2/Conv_Case1_spill.txt"
        )
    }
    output_dir = "Problem3" # 输出目录（需提前创建）
```

```

for case_name, (sched_path, graph_path, addr_path, spill_path) in
case_config.items():
    # 1. 加载问题2的结果
    # 1.1 调度序列
    with open(sched_path, 'r', encoding='utf-8') as f:
        p2_sched = [int(line.strip()) for line in f if line.strip()]
    # 1.2 节点属性
    with open(graph_path, 'r', encoding='utf-8') as f:
        p2_nodes = {n['Id']: n for n in json.load(f)['Nodes']}
    # 1.3 缓存分配
    p2_addr = {}
    with open(addr_path, 'r', encoding='utf-8') as f:
        for line in f:
            if line.strip():
                buf_id, offset = line.strip().split(':')
                p2_addr[int(buf_id)] = int(offset)
    # 1.4 SPILL列表
    p2_spill = []
    with open(spill_path, 'r', encoding='utf-8') as f:
        for line in f:
            if line.strip():
                buf_id, new_offset = line.strip().split(':')
                p2_spill.append((int(buf_id), int(new_offset)))

    # 2. 计算问题2的基准指标
    with open(graph_path, 'r', encoding='utf-8') as f:
        adj = defaultdict(list)
        for s, t in json.load(f)['Edges']:
            adj[s].append(t)
    _, _, p2_time, _ = calculate_execution_time(p2_sched, p2_nodes, adj)
    # 计算问题2的SPILL成本
    buf_lifecycle = get_buf_lifecycle(p2_sched, p2_nodes)
    p2_spill_cost = sum(
        buf_lifecycle[b][2] if buf_lifecycle[b][4] else 2 * buf_lifecycle[b]
    [2]
        for b, _ in p2_spill
    )

    # 3. 执行性能优化
    opt_sched, opt_addr, opt_spill, opt_time, opt_spill_cost =
optimize_performance(
    p2_sched, p2_nodes, p2_addr, p2_spill, p2_time, p2_spill_cost,
graph_path
    )

    # 4. 保存优化结果

```



```

        save_problem3_results(case_name, opt_sched, opt_addr, opt_spill,
output_dir)

# 5. 打印优化效果
time_reduction = (p2_time - opt_time) / p2_time * 100
spill_increase = (opt_spill_cost - p2_spill_cost) / p2_spill_cost * 100
if p2_spill_cost != 0 else 0
    print(f"【{case_name}】")
    print(f"  优化前时间: {p2_time} 周期 → 优化后: {opt_time} 周期, 降幅:
{time_reduction:.2f}%")
    print(f"  优化前SPILL成本: {p2_spill_cost} → 优化后: {opt_spill_cost}, 增
幅: {spill_increase:.2f}%")
    print("-" * 50)

```

1.4 附录

1.4.1 关键术语定义

| 术语       | 定义                                                                      |
|----------|-------------------------------------------------------------------------|
| SIMD     | 单指令多数据流架构，硬件设计简单、面效高，适用于边缘神经网络推理任务                                      |
| DAG      | 有向无环图，计算图的核心结构，节点含操作与缓存管理类型，边表示依赖关系                                     |
| SPILL 操作 | 缓存空间不足时的补救措施：- SPILL_OUT：将缓存数据搬至 DDR，释放空间- SPILL_IN：将 DDR 数据搬回缓存，重新分配地址 |
| Vstay    | 当前驻留的缓存容量，随 ALLOC 节点增加、FREE 节点减少，最大值为问题 1 的核心指标                         |
| 拓扑序      | 对 DAG 的节点线性排序，满足所有依赖边 (u , v ) 中 u 在 v 之前                               |

1.4.2 硬件架构参考

执行单元

| 单元类型   | 具体单元     | 功能                                        |
|--------|----------|-------------------------------------------|
| 计算单元   | Cube 核   | 专用于矩阵乘法计算，输入来自 L0A/L0B 缓存，结果写入 L0C 缓存     |
|        | Vector 核 | 通用向量计算（如 ADD、MUL、EXP），数据存储于 UB 缓存         |
| 数据搬运单元 | MTE1/2/3 | 在 DDR、L1、UB、L0 缓存间传输数据（如 MTE2: DDR↔L1/UB） |
|        | FIXP     | 将 Cube 核结果从 L0C 缓存写回 DDR                  |

多级缓存

| 缓存类型        | 关联单元     | 容量 (字节)     | 功能                                         |
|-------------|----------|-------------|--------------------------------------------|
| L0A/L0B/L0C | Cube 核   | 256/256/512 | L0A: 左矩阵输入缓存; L0B: 右矩阵输入缓存;<br>L0C: 结果输出缓存 |
| L1          | MTE1/2   | 4096        | 矩阵运算数据的中间缓存, 容量较大                          |
| UB          | Vector 核 | 1024        | Vector 核的输入、输出及中间结果缓存                      |

公众号: 互顶数据  
QQ群: 1061694559